

Kubernetes and Cloud Native Associate - KCNA

Scheduling

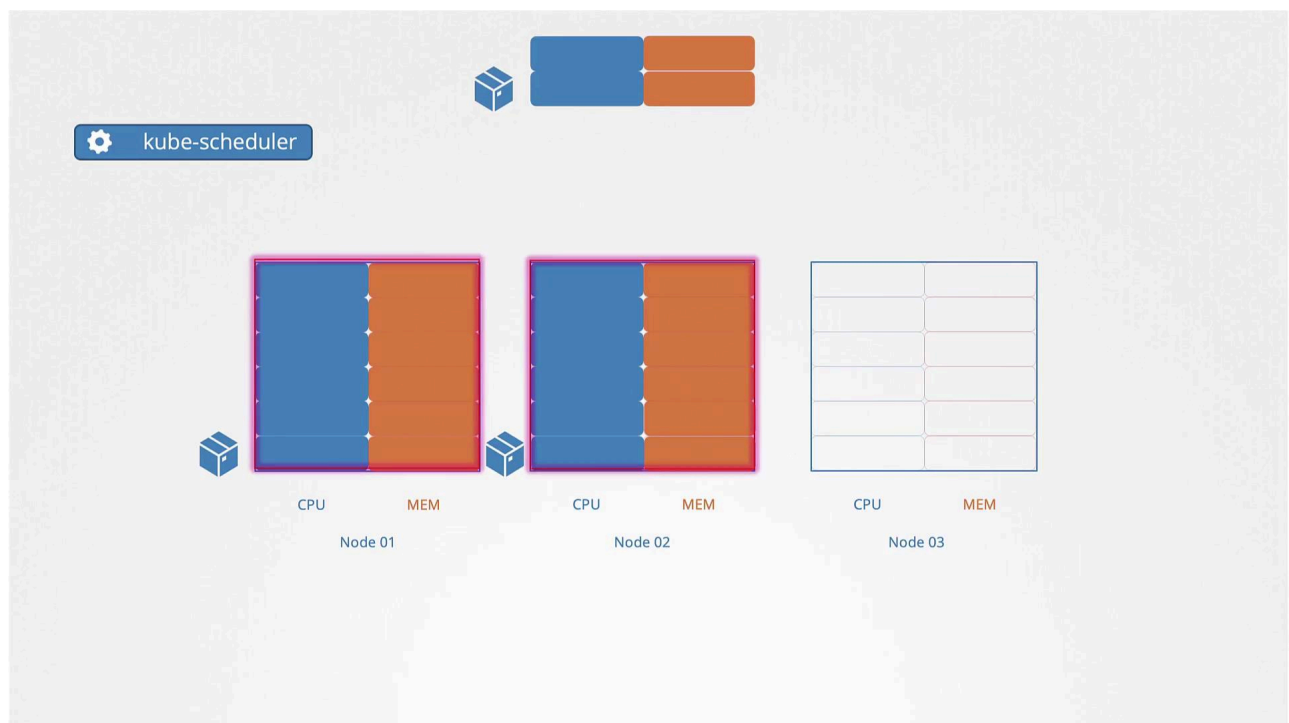
Resource Limits

In this lesson, we explore resource requirements and limits within a Kubernetes cluster. We begin with a three-node cluster architecture, where each node has a specific set of CPU and memory resources available. Every pod scheduled on a node consumes a reserved portion of these resources. For instance, a pod might require two CPUs and one unit of memory. When the pod is scheduled, it directly consumes the available resources on its host node.

The Kubernetes scheduler is tasked with selecting the optimal node for a pod by comparing the pod's resource requirements to the available capacity on each node. In the example below, the scheduler places a new pod on node two because that node meets the minimum resource criteria.



If a node does not possess sufficient resources, the scheduler bypasses it and selects an alternative node that can satisfy the pod's resource demands.



When none of the nodes meet the resource criteria, the pod remains in a pending state. Running the `kubectl describe pod` command in such cases may display an event message similar to:

NAME	READY	STATUS	RESTARTS	AGE
Nginx	0/1	Pending	0	7m

Events:

Reason	Message
-----	-----

FailedScheduling No nodes are available that match all of the following predicate

Resource Requests

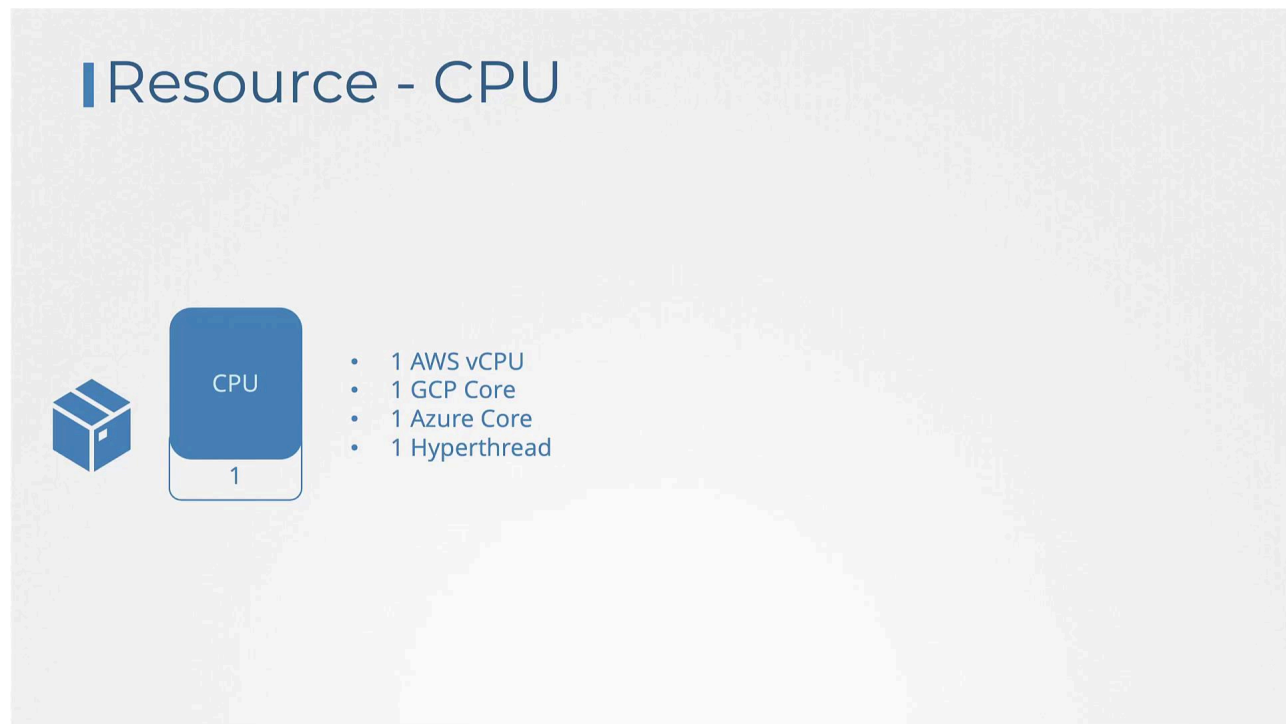
Each pod in Kubernetes can define its minimum required CPU and memory resources through resource requests. These values serve as the guaranteed baseline for the container. The scheduler uses these values to ensure that the selected node can provide at least the requested resources. Consider the pod definition example below, which requests four GB of memory and two CPU cores:

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "4Gi"
        cpu: 2
```

Kubernetes verifies that these requested resources are available on the node prior to scheduling the pod.

A few key points about CPU values:

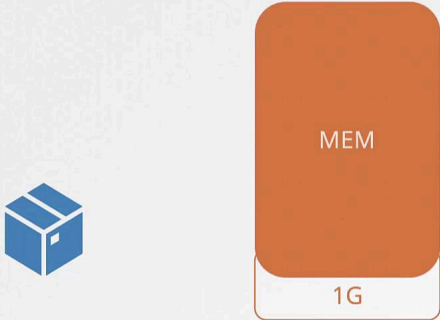
- You can specify fractional values, such as 0.1 CPU.
- The value 0.1 CPU can also be expressed as 100m (milli).
- The smallest measurable unit is 1m.
- One core of CPU is equivalent to one vCPU, as seen in most cloud providers such as AWS, GCP, and Azure, or as a hyper-thread in other systems.



Memory Requests

Memory specifications in Kubernetes can be defined using clear, concise unit suffixes. Use "Mi" for mebibytes or "M" for megabytes; similarly, use "Gi" for gibibytes or "G" for gigabytes. It is important to note that 1 G (gigabyte) equals 1000 megabytes, whereas 1 Gi (gibibyte) equals 1024 megabytes.

Resource - Memory



1 G (Gigabyte) = 1,000,000,000 bytes
1 M (Megabyte) = 1,000,000 bytes
1 K (Kilobyte) = 1,000 bytes

1 Gi (Gibibyte) = 1,073,741,824 bytes
1 Mi (Mebibyte) = 1,048,576 bytes
1 Ki (Kibibyte) = 1,024 bytes

Resource Limits

By default, containers have no upper limit on the resources they can consume. This can lead to scenarios where one container monopolizes the available resources, negatively impacting other containers and system processes. To avoid such issues, Kubernetes allows you to define resource limits for both CPU and memory.

Below is an example pod definition that sets resource limits in addition to resource requests:

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
    - name: simple-webapp-color
      image: simple-webapp-color
      ports:
        - containerPort: 8080
```

```
resources:
  requests:
    memory: "1Gi"
    cpu: 1
  limits:
    memory: "2Gi"
    cpu: 2
```

When the pod starts, Kubernetes allocates the specified resources for each container. If the pod consists of multiple containers, each container should have clearly defined resource requests and limits.



Note

While CPU limits restrict a container from surpassing its assigned CPU capacity by throttling, memory limits function differently. A container may temporarily exceed its set memory limit; however, if it does so consistently, the system will terminate the container with an Out Of Memory (OOM) error.

Exceed Limits



Default Behavior and Best Practices

By default, Kubernetes does not enforce resource requests or limits. As a consequence, a pod that lacks these settings may consume all available resources on its host node, potentially starving other pods.

CPU Resource Scenarios

1. **No requests or limits set:** A pod may consume all CPU resources, adversely affecting other deployments.
2. **Limits set but no requests:** Kubernetes assigns the request value equal to the limit. For example, if the limit is set to 3 vCPUs, the pod is guaranteed 3 vCPUs.
3. **Both requests and limits set:** A pod is guaranteed its requested amount (e.g., 1 vCPU), but it can scale up to the defined limit (e.g., 3 vCPUs) if resources permit.
4. **Requests set but no limits:** The pod is guaranteed its requested CPU (e.g., 1 vCPU) and can consume any unutilized CPU cycles on the node, offering a balanced approach to resource efficiency.

Memory Resource Scenarios

Memory management follows a similar concept:

1. **Without requests or limits:** A pod may consume all available memory, potentially leading to instability.
2. **Only limits specified:** Kubernetes assigns the pod's memory request equal to its limit (e.g., 3 GB).
3. **Both requests and limits specified:** The pod is guaranteed its requested memory (e.g., 1 GB) and can utilize up to the limit (e.g., 3 GB).
4. **Requests set without limits:** The pod is guaranteed its requested memory; however, unlike CPU, exceeding this value may lead to the pod being terminated if system memory runs low.



Warning

Always set resource requests for your pods. A pod without specified requests can overconsume resources, potentially leading to performance degradation of other pods with defined limits.

Limit Ranges

Resource requests and limits are not automatically applied to pods in Kubernetes. To enforce default resource configurations, use LimitRange objects at the namespace level. This ensures that pods without explicit resource definitions receive predefined values.

Below is an example LimitRange for managing CPU resources:

```
# limit-range-cpu.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-resource-constraint
spec:
  limits:
  - default:
      cpu: 500m
    defaultRequest:
      cpu: 500m
    max:
      cpu: "1"
    min:
      cpu: 100m
    type: Container
```

Likewise, you can define a LimitRange for memory as follows:


```
# limit-range-memory.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: memory-resource-constraint
spec:
  limits:
  - default:
      memory: 1Gi
    defaultRequest:
      memory: 1Gi
    max:
      memory: 1Gi
    min:
      memory: 500Mi
    type: Container
```

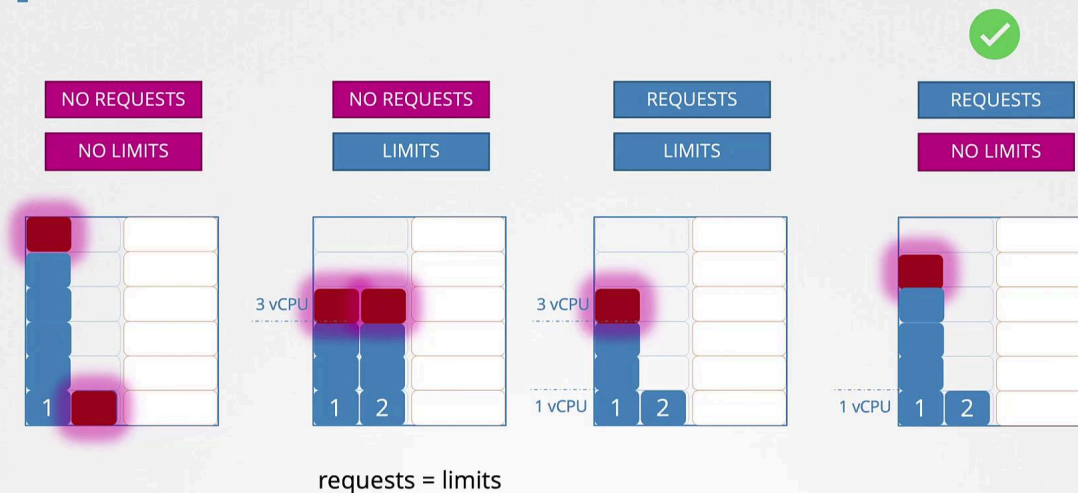
Note that any modifications to LimitRange objects affect only pods that are created after the changes have been applied.

Resource Quotas

ResourceQuota objects enable administrators to limit the overall resources that applications in a namespace can consume. By setting a ResourceQuota, you can restrict the total requested CPU and memory for all pods within the namespace. For example, you may limit the total requested CPU to 4 vCPUs and total requested memory to 4 GB, while imposing a maximum limit of 10 vCPUs and 10 GB of memory across all pods combined.

This strategy is particularly useful in environments such as public labs, where managing resource utilization is crucial to avoid infrastructure abuse.

Behavior - CPU



For additional information on managing CPU, memory, and overall API resources, refer to the [Kubernetes Documentation](https://kubernetes.io/docs/tasks/administer-cluster/).

Documentation References

Manage Memory, CPU and API Resources

<https://kubernetes.io/docs/tasks/administer-cluster/manage-resources/>

LimitRange for CPU

<https://kubernetes.io/docs/tasks/administer-cluster/manage-resources/cpu-default-namespace/>

LimitRange for Memory

<https://kubernetes.io/docs/tasks/administer-cluster/manage-resources/memory-default-namespace/>



Watch Video

Watch video content

Previous

◀ Taints and Tolerations vs Node Affinity

Next

DaemonSets ▶